

Zadanie Ośrodek Narciarski – Kolejny Sezon

Pierwsza wersja symulacji spotkała się z pozytywnym przyjęciem ze strony ośrodka narciarskiego. Projekt posłużył już do wykonania kilku wstępnych analiz planu przebudowy. Przedstawiciele ośrodka zwrócili uwagę na ograniczenia symulacji i zgłosili zapotrzebowanie na wprowadzenie ulepszeń.

Sportowcy

Okazuje się, że rzeczywiste zachowanie sportowców jest znacznie bardziej skomplikowane niż dotychczas przypuszczano. Oprócz sportowców podejmujących decyzje tak, jak w pierwszej części zadania, wyróżniono jeszcze dwa nowe rodzaje sportowców.

Plany przejazdów

Sportowcy opisani poniżej układają plany przejazdu do wybranych przez siebie tras. W tym celu przeszukują graf wszystkich połączeń (wyciągów i tras) i wybierają jak najkrótszy ich ciąg prowadzący do pożądanego trasy. Jeśli ciągów tej długości jest wiele, wybierają dowolny. Taki plan realizują krok po kroku aż dotrą do wymarzonej trasy i nią zjadą. Dopiero wówczas zaczynają podejmować kolejne decyzje. Zmiany planu oraz spontaniczna decyzja są możliwe tylko gdy poprzedni plan został w całości zrealizowany.

Sportowcy starego typu zachowują się pod tym względem tak samo jak dotychczas, a więc mogą zmienić zdanie przed zjechaniem wybraną trasą.

Do wyznaczenia takiego planu można użyć algorytmu BFS. W celu odtworzenia ścieżki w algorytmie BFS należy, poza pamiętaniem, czy wierzchołek został już przetworzony i jaka jest jego odległość, zapisać też odpowiednią wchodzącą do niego krawędź (tą którą został znaleziony po raz pierwszy). Na koniec algorytmu ścieżkę można odzyskać od tyłu, cofając się po krawędziach od wierzchołka docelowego.

Nowe rodzaje sportowców

Zachłanny

Zachłanny sportowiec upatruje sobie najatrakcyjniejszą trasę spośród tras dostępnych w całym ośrodku. Układa plan przejazdu do tej trasy.

Kolekcjoner

Ten typ sportowca przede wszystkim stara się odwiedzić jak najwięcej tras. W tym celu układa plan przejazdu do trasy którą jeszcze nie zjeżdżał (dokładniej: zjeżdżał minimalną liczbę razy). Remisy rozstrzyga najpierw według odległości - rozumianej w uproszczeniu jako liczba tras i wyciągów którymi trzeba przejechać by do danej trasy dotrzeć, a w drugiej kolejności według atrakcyjności. Jeżeli to nie wystarcza do rozstrzygnięcia, wybiera dowolne spośród remisujących rozwiązań.

Znudzenie trasą

Analitycy ośrodka zauważyli, że sportowcy z czasem nudzą się trasami, którymi zjeżdżają. Zjawisko to modelujemy **poziomem znudzenia** z_t o wartościach $[0, 1]$ - wartością określającą jak bardzo dany sportowiec znudzony jest daną trasą w obecnym momencie.

Do modelowania znudzenia używamy mechanizmu wykładniczego. Przyjmujemy że początkowe znudzenie to zawsze $z_0 = 0$: gdy zaczyna się dzień wszyscy sportowcy są pełni entuzjazmu. Kolejna wartość wyliczana jest w momencie zjazdu przez sportowca dowolną trasą według wzoru

$$z_t = \beta \cdot x_t + (1 - \beta)z_{t-1},$$

gdzie x_t określa czy sportowiec zjeżdża właśnie rozważaną trasą (1 jeśli zjeżdża daną trasą, 0 jeśli jakąkolwiek inną), a β to współczynnik znudzenia z przedziału $[0, 1]$ przybliżający zachowanie pamięci sportowca.

Dopuszczamy tutaj skrajne wartości β : 0 oznacza (zgodnie ze wzorem) że sportowiec nigdy się nie nudzi, a 1 że bardzo nie lubi powtarzać tej samej trasy dwa razy z rzędu.

Zauważmy że nie trzeba przeliczać tych wartości za każdym razem gdy sportowiec zjeżdża jakąś trasą - jeśli pamiętamy indeks k ostatniego zjazdu daną trasą i wartość z_k dla tej trasy, to po i przejazdach sportowca innymi trasami wartość z_{k+i} wyniesie $z_k \cdot (1 - \beta)^i$.

Wzór na atrakcyjność należy uzupełnić o znudzenie.

Nowy wzór na atrakcyjność

$$a(p_n, p_t, k_t, b_t, o_t, z) = \alpha_d \cdot d(p_n, p_t) + \alpha_w \cdot w(k_t, b_t, o_t) + \alpha_z \cdot (1 - z)$$

Gdzie:

- z - obecne znudzenie sportowca daną trasą,
- $\alpha_d, \alpha_w, \alpha_z$ - nieujemne wagi określonych aspektów (indywidualne dla każdego sportowca), sumujące się do 1,
- $d, w, p_n, p_t, k_t, b_t, o_t$ - wyjaśnione w treści pierwszej części zadania.

Nowy format danych wejściowych

Z powodu tych zmian zachodzi konieczność modyfikacji formatu danych wejściowych dotyczących sportowców.

Nowy opis grupy sportowców (w podziale na linie) wygląda następująco:

1. Liczbę sportowców w grupie (dodatnią), poziom zaawansowania, współczynnik spontaniczności, **współczynnik znudzenia**, **oznaczenie rodzaju sportowców** i, opcjonalnie literę **s**, jeśli sportowcy z grupy mają być śledzeni.
2. Wagi atrakcyjności poszczególnych aspektów tras, kolejno: dopasowania poziomem, wyrównania nawierzchni oraz **znudzenia**.
3. Numer startowego węzła, godzinę rozpoczęcia dnia na stoku przez pierwszego sportowca z grupy (w formacie HH:MM:SS) oraz, jeśli jest ich więcej niż jeden, czasowy odstęp między początkami dnia kolejnych sportowców (jako całkowita nieujemna liczba sekund).

Oznaczenie rodzaju sportowca to jedna litera: L - Lokalny (sportowiec z poprzedniej wersji), Z - Zachłanny, K - Kolekcjoner. Współczynnik znudzenia to β opisana powyżej. Waga znudzenia α_z to czynnik określający istotność znudzenia dla danego sportowca.

Dwa zestawy przykładowych danych w nowym formacie są dołączone do zadania w Moodle'u.

Kontenery

W nowej wersji symulacji dozwolone jest korzystanie z biblioteki standardowej Javy, w tym kontenerów, typów uogólnionych, strumieni oraz lambd.

Należy podmienić implementację kolejki zdarzeń na biblioteczną kolejkę priorytetową. Aby zachować stabilność (zdarzenia o równym czasie kolejka powinna wydawać w kolejności ich wstawiania) przydatne może być opakowanie zdarzenia w parę wraz z numerem porządkowym i przechowywanie w kolejce takich par. W tym celu można przygotować odpowiednią klasę (w Javie brakuje wbudowanych Par) i komparator.

Statystyki

By zaspokoić potrzeby rosnącego zespołu analityków ośrodka, potrzeba dostarczyć nowych statystyk o jego funkcjonowaniu. W ostatecznym raporcie powinny znaleźć się następujące statystyki podsumowujące dzień w ośrodku.

Trasy

Dla każdej trasy należy określić:

1. łączną liczbę przejazdów,
2. wyrównanie trasy na koniec dnia.

Wyciągi

Dla każdego wyciągu należy określić:

1. maksymalną długość kolejki,
2. średnią długość kolejki (średnią liczoną po wszystkich sekundach symulacji; w danej sekundzie interesuje nas długość kolejki po przetworzeniu wszystkich zdarzeń dotyczących tej sekundy),
3. łączną liczbę przewiezionych pasażerów,
4. procent zajętych miejsc (łączna liczba przewiezionych pasażerów / maksymalna możliwa łączna liczba pasażerów których wyciąg mógłby przewieźć).

Zauważmy, że sumy długości kolejki (potrzebnej do wyliczenia średniej długości kolejki) nie musimy (i nie powinniśmy) aktualizować co sekundę. Wystarczy pamiętać godzinę ostatniej operacji wykonanej na kolejce. Przy kolejnej operacji aktualizujemy sumę o wszystkie sekundy od czasu poprzedniej operacji (długość kolejki była przez ten czas stała).

Wizualizacja

Czytając statystyki w formie tekstowej, trudno dostrzegać przestrzenne zależności. Analitycy poprosili więc o generowanie wizualizacji przedstawiających najważniejsze dla nich informacje na mapie ośrodka.

Generator mapek

Zestaw klas do generowania mapek ośrodka jest już zaimplementowany (należy pobrać go z Moodle'a i dołączyć go jako część kodu rozwiązania). Państwa zadaniem jest użycie tych klas tak, żeby symulacja generowała mapki przedstawiające odpowiednie dane.

Przykładowe użycie:

```
// Tworzy generator i nowy katalog, w którym generator będzie zapisywał
// wszystkie mapki.
GeneratorMapek generatorMapek = new GeneratorMapek("katalog/do/mapek");

// Argumenty: numer, x, y, styl
generatorMapek.dodajWezel(0, 2, 3, new StylWezla(GruboscKonturu.POGRUBIONY));

generatorMapek.dodajWezel(1, 10, 7, new StylWezla(GruboscKonturu.ZWYKLY));

ArrayList<String> linieTekstu = new ArrayList<>();
linieTekstu.add("Pierwsza linia");
linieTekstu.add("Druga linia");
// Argumenty: węzeł początkowy, końcowy, styl, tekst.
generatorMapek.dodajKrawedz(0, 1, new StylKrawedzi(StylLinii.PRZERYWANA), linieTekstu);

generatorMapek.zapiszMapke("nazwa-pliku.tex");

// Jeśli chcemy zacząć tworzyć nową mapkę:
generatorMapek.zeruj();
// I od nowa dodajemy węzły i krawędzie.
```

Szczegółowy opis każdej z metod jest w pliku `GeneratorMapek.java`.

Z gotowych plików TeX można generować obrazki w formacie PDF następującym poleceniem:

```
$ pdflatex nazwa-pliku.tex
```

Polecenie działa na komputerze students. Inną wygodną możliwością jest serwis Overleaf lub zainstalowanie LaTeXa lokalnie na własnym komputerze – więcej informacji na końcu treści zadania.

Państwa program powinien wygenerować tylko pliki TeX. Dalszą konwersję do PDF (wspomnianym poleceniem) zajmą się sprawdzający. Należy przy tym upewnić się, że pliki TeX generowane przez Państwa programy konwertują się poprawnie (tym poleceniem) do PDF na komputerze students i że wynikowy obrazek faktycznie prezentuje wymagane informacje.

Rodzaje mapek

Parametry tras i wyciągów

Pierwsza mapka ma przedstawiać graf węzłów, tras i wyciągów – oraz ich parametry. Współrzędne węzłów podane są na wejściu programu (patrz: format wejścia z pierwszej części zadania). Linie tras mają być ciągłe, a wyciągów przerywane. Kontur węzłów skomunikowanych ma być pogrubiony względem pozostałych węzłów.

Każda trasa powinna prezentować swoje parametry w następującym formacie (dane przykładowe):

```
t1: poziom: 7, czas: 180s
odporność: 0.30, 0.99970
```

`t1` odpowiada numerowi trasy. `0.30` to bazowa atrakcyjność niewyrównanej trasy (z dokładnością do 2 miejsc po kropce dziesiętnej), a `0.99970` to odporność trasy przeciw tworzeniu się nierówności (dokładność 5 miejsc po kropce). Akceptujemy też użycie przecinka jako separatora dziesiętnego (i w mapkach i w wypisywaniu na wyjście).

Każdy wyciąg powinien prezentować swoje parametry w następującym formacie (dane przykładowe):

w0: 4 os. co 10s
czas: 240s

Przykład pokazuje wyciąg o numerze 0, który zabiera 4 osoby co 10 sekund, a podróż z dołu na górę zajmuje 240s.

Plik z przykładową mapką tego rodzaju (odpowiadającą pierwszemu plikowi z przykładowymi danymi wejściowymi) jest dołączony do zadania w Moodle'u.

Dopisek: Zgodnie z duchem PO dobór formatu wypisywania danych powinniśmy pozostawić Państwu. Natomiast na krawędziach wynikowego grafu w czytelny sposób mieści się dość mało tekstu (empiryczny sensowny limit linii to ok. 30 znaków). Narzucamy więc możliwie zwięzły a przy tym intuicyjny format, który udało nam się wymyślić. Dzięki temu możemy zmieścić na mapce więcej informacji, które mogą dać ciekawsze obserwacje.

Statystyki tras i wyciągów

Druga mapka ma przedstawiać graf ośrodka ze statystykami na temat każdej trasy i wyciągu. Układ i formatowanie tej mapki jest identyczne, jak w przypadku poprzedniej mapki. Różnica tkwi w tekście wypisywanym na krawędziach grafu.

Statystyki tras i ich format:

t1: śnieg: 0.23
zjazdy: 5191

Przykład pokazuje trasę numer 1, której atrakcyjność związana z wyrównaniem nawierzchni na koniec symulacji wyniosła 0.23 (2 miejsca po kropce). W trakcie całej symulacji trasą tą zjechało 5191 razy.

Statystyki wyciągów i ich format:

w0: kol: 68(śr), 150(maks)
wjazdy: 10040 / 10400 (96%)

Przykład pokazuje wyciąg numer 0. W kolejce do tego wyciągu oczekiwało średnio 68 osób (zaokrąglamy do najbliższej liczby całkowitej), a w szczytowym momencie było ich aż 150. W czasie całej symulacji wyciągiem tym wjechało 10040 razy, co stanowi ok. 96% jego przepustowości (10400 to maksymalna możliwa liczba wjazdów w ciągu dnia tym wyciągiem).

Historia przejazdów sportowca

Trzecia mapka (a w zasadzie rodzaj mapek) pokazuje kolejne trasy i wyciągi, którymi śledzony sportowiec poruszał się w trakcie symulacji. Układ i formatowanie tej mapki jest takie samo, jak we wcześniejszych mapkach – i różnica znów polega tylko na tekście krawędzi.

Na potrzeby tej mapki, koncepcyjnie, numerujemy (od jedynki) wszystkie przejazdy sportowca trasami i wyciągami, zgodnie z ich czasową kolejnością. Numeracja przejazdów trasami i wyciągami jest wspólna. Na przykład jeśli sportowiec najpierw wjechał wyciągiem nr 5, potem zjechał trasą nr 8, potem znów wjechał wyciągiem nr 5 i zjechał trasą nr 8, to wjazdy wyciągiem nr 5 mają numery 1 i 3, a zjazdy trasą nr 8 mają numery 2 i 4.

Na każdej trasie i wyciągu chcemy wypisać liczbę przejazdów tego sportowca tą trasą lub wyciągiem oraz ciąg numerów przejazdów tego sportowca tą trasą lub wyciągiem.

Przykład i format dla trasy:

t3(2): 3,10

Sportowiec trasą numer 3 zjechał 2 razy – i był to jego trzeci i dziesiąty przejazd tego dnia.

Przykład i format dla wyciągu:

```
w1(29): 1,4,7,13,16,19,22,25,  
28,31,34,37,40,46,49,52,55,58...
```

Sportowiec bardzo polubił wyciąg numer 1 (lub pewnie trasę do której ten wyciąg prowadzi) – wjechał nim aż 29 razy. Liczba przejazdów była na tyle duża, że wszystkie ich numery nie zmieściły się na krawędzi – i `GeneratorMapek` uciął nadmiarowe przejazdy (sygnalizuje to końcowy wielokropek).

W przypadku tej mapki należy skorzystać z drugiej (przeciążonej) wersji metody `GeneratorMapek::dodajKrawedz`, która sama dzieli tekst między dwie linie i odpowiednio go przycina, jeśli uzna to za konieczne (szczegóły w dokumentacji metod w pliku `GeneratorMapek.java`). Użytkownik metody musi tylko podać odpowiedni (być może bardzo długi) tekst, a metoda zajmie się resztą.

Mapkę z historią tras należy wygenerować dla każdego śledzonego sportowca (będzie więc to zwykle kilka mapek). Nazwa pliku każdej takiej mapki powinna zawierać (w szczególności) numer sportowca, którego dotyczy.

Uwagi techniczne wizualizacji

Katalog na mapki

Wszystkie mapki z pojedynczego uruchomienia programu powinny znaleźć się w jednym katalogu. Stąd też (jak czytamy się w dokumentację `GeneratoraMapek`), wszystkie mapki należy wygenerować tym samym (jedynym) obiektem generatora.

Ścieżkę do katalogu, który program ma utworzyć i zapisywać tam mapki, podaje użytkownik – poprzez argument programu (można odczytać go z tablicy `String[] args` w metodzie `main`). W terminalu takie uruchomienie wygląda następująco:

```
$ java -ea pakiet.GłównaKlasa "ściezka/katalogu/do/mapek" < dane-wejscowe.txt
```

Argumenty programu i JVM da się również ustawić, pracując nad rozwiązaniem w zintegrowanym środowisku programistycznym, ustawienie tam przekierowania może być trudniejsze.

Może się zdarzyć, że użytkownik programu zapomni o podaniu ścieżki do katalogu. Program powinien poprawnie obsłużyć taką sytuację – należy wypisać odpowiedni komunikat na wyjście diagnostyczne i zakończyć program. Może też się zdarzyć, że użytkownik poda niepoprawną ścieżkę (w której np. nie da się utworzyć katalogu ze względu na brakujące uprawnienia). Nie muszą Państwo sami implementować weryfikacji ścieżki podanej przez użytkownika – klasa `GeneratorMapek` zawiera odpowiednią logikę i w razie problemów zgłosi wyjątek. Państwa zadaniem jest poprawne obsłużenie takiego wyjątku (więcej o tym w sekcji *Obsługa wyjątków*).

Jak zagwarantować, że mapka będzie czytelna?

Innymi słowy: *Jak mogę zagwarantować, że każda krawędź będzie na tyle długa, żeby jej tekst nie zasłonił sąsiedniego węzła? Albo że tekst z jednej krawędzi (leżącej blisko innej krawędzi), nie przesłoni części tekstu tej drugiej?*

Odpowiedź: Z poziomu Państwa programów niestety się nie da (lub byłoby to bardzo trudne). Odpowiedzialność za układ graficzny mapki na danym teście (i to, żeby wymagany tekst czytelnie się w tym układzie wyświetlał) leży po stronie autora testu (czytaj: autorów zadania). Państwa zadaniem jest wywołanie właściwych metod `GeneratoraMapek` z właściwymi danymi symulacji

(opisanymi wcześniej). Jeśli obsługują Państwo `GeneratorMapek` poprawnie, a mimo to niektóre fragmenty mapki wyświetlają się tak sobie, to:

1. warto upewnić się, że rzeczywiście dobrze obsługują Państwo `GeneratorMapek`, a w drugiej kolejności
2. nie przejmować się tym – widocznie na danym teście tak wychodzi.

Przy czym autorzy zadania starali się, żeby przestrzenny układ ośrodków narciarskich z testów rysował się przejrzysto `GeneratoremMapek` – więc liczymy na to, że nie będzie z tym problemów.

Wydaje mi się, że w `GeneratorzeMapek` jest błąd. Co z tym zrobić?

Prosimy zgłaszać takie problemy na forum. Jeśli faktycznie jest błąd, poprawimy.

Obsługa wyjątków

`GeneratorMapek` zgłasza `WyjatekSystemuPlikow`, gdy napotka problem z obsługą systemu plików – np. nie uda się utworzyć katalogu czy zapisać pliku z mapką. Takie sytuacje są zwykle poza kontrolą programu – np. użytkownik podał ścieżkę, do której nie ma dostępu, lub skończyło się miejsce na dysku (do tego drugiego nie powinno dojść; pliki mapek są dość małe). Zgłoszenie wyjątku `WyjatekSystemuPlikow` powinno przerwać symulację i sterowanie programem powinno wrócić do głównej metody programu. Program powinien wtedy wypisać komunikat na wyjście diagnostyczne, który zasugeruje użytkownikowi, żeby upewnił się, że podał poprawną ścieżkę i że ma odpowiednie uprawnienia. Dzięki temu użytkownik będzie mógł odpowiednio zareagować (np. znaleźć lepszą lokalizację na katalog na mapki) i uruchomić program ponownie.

Wystąpienie innych wyjątków świadczy prawdopodobnie o błędzie programisty w logice programu (np. podanie niepoprawnego argumentu do metody, czy dzielenie przez zero). W takiej sytuacji należy również przerwać symulację, wrócić do głównej metody programu i wypisać komunikat na wyjście diagnostyczne, który tym razem zasugeruje użytkownikowi, żeby zgłosił błąd zespołowi deweloperskiemu (w tym przypadku: Państwu).

W jednym i w drugim przypadku za sugestią dla użytkownika należy wypisać na wyjście diagnostyczne ślad stosu (ang. stack trace) związany z linią kodu, w której został zgłoszony wyjątek (w klasie `Throwable` jest odpowiednia do tego metoda). Ślad stosu ułatwi programiście zlokalizowanie błędu, a użytkownikowi może też nieco więcej zasugerować, w czym leży problem (przy czym w przypadku projektów z niejawnym kodem raczej nie eksponuje się śladów stosu).

W jednym i w drugim przypadku program powinien zakończyć się poprawnie, w kontrolowany sposób (a nie np. niezłapanym wyjątkiem). Nie należy korzystać z metod w rodzaju `System.exit()`.

Testy

Napisanie testów dobrze sprawdzających funkcjonalności programu jest czasochłonnym zadaniem – zwykle porównywalnym czasowo z napisaniem samej funkcjonalności. W tym zadaniu chcemy poświęcić najwięcej uwagi aspektom implementacji funkcjonalności, więc ograniczymy się do częściowego zestawu testów. Wymagamy zrealizowania za pomocą **JUnit** następujących scenariuszy:

Odjazdy wyciągów

Należy przetestować następujące scenariusze, typowe dla pracy wyciągu:

1. Jest wyciąg o pojemności 3 z pustą kolejką. Ustawia się do niego czterech sportowców. Następuje odjazd wagonika. Należy sprawdzić, że przejazd wykonało trzech sportowców (jeden z nich się nie zmieścił) - licznik odjazdów wyciągu zwiększył się o 3.
2. Do wyciągu o pojemności 3 ustawia się dwóch sportowców, następuje odjazd wagonika. Należy sprawdzić że obydwaj odjechali.
3. Do wyciągu ustawia się kolejno czterech sportowców. Odjeżdża wagonik, a następnie ustawia się jeszcze jeden. Należy sprawdzić że poprawnie obliczono maksymalną długość kolejki - 4.

Przeszukiwanie grafu

Należy przetestować działanie metod wyszukujących najkrótsze ścieżki w grafie.

Zwykle nie należy eksponować prywatnych metod klasy tylko po to, żeby je przetestować – zachowanie klasy testuje się przez jej publiczny interfejs (w znaczeniu *zestaw publicznych metod*; niekoniecznie konstrukcja Javy). Szukanie najkrótszej ścieżki jest szczególnie implementacyjnym sportowca (z perspektywy innych klas ważna jest tylko ostateczna decyzja), więc sportowiec nie powinien takich metod eksponować (kapsułkowanie). Ale sprawdzanie poprawności BFSa po samej ostatecznej decyzji sportowca byłoby trudne.

Wyznaczanie najkrótszych ścieżek stanowi na tyle nietrywialną logikę, że zasługuje na osobną klasę (z odpowiednimi publicznymi metodami). Taką klasę możemy już wygodnie przetestować. A sportowiec może skorzystać z takiej (przetestowanej) klasy na własne potrzeby, nie chwając się tym na zewnątrz. Nawiasem mówiąc, łatwość testowania jest jednym z kryteriów, które warto mieć na uwadze, projektując klasy programu.

W teście należy utworzyć graf i przetestować poprawność wyliczania odległości i znajdowania najkrótszej ścieżki przez klasę do przeszukiwania grafu.

Algorytm należy przetestować na grafie o strukturze jak na obrazku po prawej stronie (dokładne parametry tras i wyciągów nie mają znaczenia).

Należy sprawdzić że algorytm poprawnie oblicza ścieżki i odległości:

1. Ścieżkę z wierzchołka 0 do 4: prowadzącą poprzez 0, 1, 2, 4, odległość 3.
2. Bezpośrednią ścieżkę z 3 do 1.
3. Pustą ścieżkę z 2 do samego siebie (odległość 0).
4. Ścieżkę z 4 do 3: algorytm wybiera jedną z dwóch możliwych ścieżek długości 2.

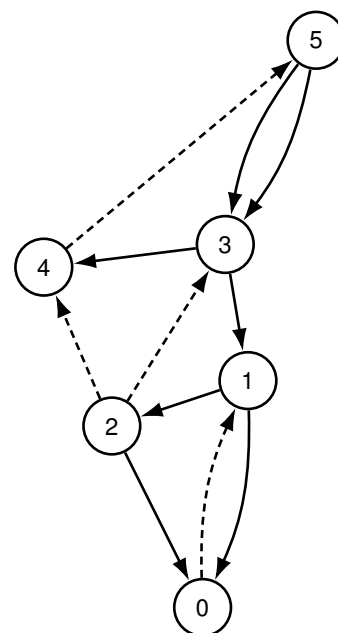


Figure 1: Graf do testu przeszukiwania. Jest to (nieco zmodyfikowany) fragment ośrodka z drugiego pliku z przykładowymi danymi.

Dodatkowe uwagi

Rozwiązanie pierwszego zadania

Istotną wartością podwójnego zadania (zamiast dwóch niezależnych zadań) jest to, że mają Państwo okazję zobaczyć, jak pracuje się z kodem większego projektu, który wcześniej sami Państwo napisali. To szansa empirycznego sprawdzenia, które z podjętych wcześniej decyzji projektowych czy sposobów implementacji poszczególnych klas faktycznie pomagają w dalszym rozwijaniu projektu, a które to zadanie utrudniają. Praca przez dłuższy czas nad jednym projektem, za który się odpowiada, jest także typowym doświadczeniem w pracy programisty – więc takie zadanie mogą Państwo potraktować jako pewnego rodzaju namiastkę przyszłej pracy zawodowej.

Wiemy natomiast, że niektórzy z Państwa mogą czuć się niepewnie, jeśli chodzi o jakość swoich rozwiązań pierwszego zadania i możliwości ich dalszego rozwijania. Z myślą o takich osobach zamieszczamy w Moodle'u nasze rozwiązanie pierwszego zadania, które można skopiować i użyć jako punkt wyjścia (zamiast własnego rozwiązania pierwszego zadania) do pracy nad drugą częścią zadania. Nie będzie to miało żadnych konsekwencji punktowych – takie prace będziemy oceniać tak samo, jak kontynuacje własnych rozwiązań części pierwszej.

To powiedziawszy, zachęcamy Państwa do kontynuacji własnych rozwiązań pierwszego zadania i skorzystania w pełni z walorów podwójnego zadania.

Ogólne

- Kod większych programów wymaga refaktoryzacji. Często dopiero po napisaniu działającego programu zauważa się możliwe usprawnienia kodu, poprawiające czytelność i jakość kodu (typowy przykład – po napisaniu rozwiązania zauważa się, że dwie metody mają podobny fragment kodu, który można, a nawet należy, wydzielić w postaci osobnej metody). Ta faza tworzenia oprogramowania wymaga czasu. Prosimy więc nie zostawiać pisania programów na ostatnią chwilę.
- Rozwiązanie ma działać na komputerze students. Students ma Javę w wersji 21 – więc takiej też wersji warto używać, pracując nad zadaniem.
- Java domyślnie nie sprawdza asercji. Przy pracy nad rozwiązaniem warto mieć je włączone (przy ocenianiu rozwiązań będą włączone). Asercje włącza się argumentem `-ea` polecenia `java` (np. IntelliJ ten argument można ustawić tutaj: **Edit Configurations > Modify options > Add VM options**).

Jak generować pliki PDF mapek?

Komputer students

Można przysyłać wygenerowane na własnym komputerze pliki `.tex` na komputer students (np. poleceniem `scp`), na studentsie generować PDF (poniższym poleceniem) i wynikowe pliki PDF przysyłać z powrotem na własny komputer (znów przez `scp`), żeby móc je obejrzeć.

Polecenie do generowania PDF:

```
$ pdflatex nazwa-pliku.tex
```

Taki tryb pracy może być niewygodny. Warto rozważyć jedną z opcji w kolejnych sekcjach.

Overleaf

Można założyć bezpłatne konto w serwisie <https://www.overleaf.com>. Tworzymy nowy projekt (raz), ładujemy wygenerowane pliki `.tex` do projektu (przeciągnięcie pliku do okna przeglądarki działa), tam generujemy PDF z mapką i na koniec pobieramy na komputer (lub oglądamy w serwisie Overleaf bez konieczności pobierania).

To wygodna opcja o praktycznie zerowym koszcie wejścia.

Instalacja LaTeXa na własnym komputerze

Można zainstalować jedną z dystrybucji LaTeXa na własnym komputerze. Pliki PDF generujemy takim samym poleceniem jak w sekcji *Komputer students*. Wszystko robimy na własnym komputerze i nie musimy przenosić żadnych plików.

Ta opcja ma niezerowy (choć nie duży) koszt wejścia – instalacja LaTeXa powinna być dość prosta, przy czym sam czas instalacji może być krótszy lub dłuższy (w zależności od systemu, dystrybucji LaTeXa, komputera). Z drugiej strony zainstalowany na komputerze LaTeX może przydać się później w trakcie studiów – na matematycznych przedmiotach czy przy pisaniu pracy licencjackiej/magisterskiej. Przy czym Overleaf jest wciąż mocną alternatywą, która na potrzeby studiów może w zupełności wystarczyć.

Instrukcja instalacji LaTeXa na najpopularniejszych systemach operacyjnych:
<https://lk.mimuw.edu.pl/pl/helpdesk-instytutow/tex>